Students:
Mostafa Ahmed [u6111119@udg.edu]
Houssam korib [u6106030@udg.edu]

Lab2B: Robot Manipulation – Pick and Place with the Stäubli TX60

Nov 10, 2025

1 Task Description

The objective of this laboratory session is to program and execute a complete pick-and-place assembly process using the Stäubli TX60 industrial robot within the Stäubli Robotics Suite (SRS) and the VAL3 programming environment.

The task consists of manipulating two mechanical components: a red cylinder and a black piston. The robot must pick the red cylinder from the pneumatic dispenser, pick a piston from the source tray, assemble both parts at the assembly station, and finally place the assembled component in the final storage matrix.

The lab is structured into three incremental levels, each adding complexity and realism to the robotic process:

- **Level 1:** Basic assembly sequence. The robot performs a simple pick-and-place routine: pick the cylinder from the dispenser, pick the piston from the tray, assemble them at the station, and place the result on the final ramp.
- **Level 2:** Continuous feeding. The robot takes control of the cylinder dispenser via a pneumatic actuator, ensuring cylinders are always available. It repeats the full assembly sequence four times while placing completed parts into four predefined matrix locations.
- **Level 3:** Continuous feeding with disturbances. A presence sensor located on the tool is introduced. Some piston tray positions are intentionally left empty, requiring the robot to detect whether a piston exists before picking. If the current slot is empty, the robot must automatically move to the next available one before continuing the assembly.

By the end of this laboratory task, the robot executes a fully autonomous assembly cycle: feeding cylinders using the dispenser actuator, detecting the presence of pistons, assembling both components, and placing each completed assembly into the correct matrix position while responding to missing-part conditions in real time.

Program Breakdown

The developed VAL3 program controls the Stäubli TX60 robot to execute an automated assembly process consisting of cylinder dispensing, piston picking, part assembly, and final matrix placement. The program makes extensive use of joint-space motions (`movej`) combined with digital I/O signals to actuate the gripper, control the cylinder dispenser, and read feedback from presence sensors. The logic is structured to follow the laboratory requirements for Levels 1, 2, and 3, enabling safe and repeatable manipulation of the red cylinder and the black piston.

Note: Two different tool frames are used during execution. `Tool0[0]` corresponds to the lateral gripper used for handling the red cylinder, while `Tool0[1]` corresponds to the tip gripper used for handling the black piston. Approach points (`up*`) and operation points (`red*`, `black*`, `together*`, `matrix*`) were named according to the provided simulation environment in Stäubli Robotics Suite.

Motion Sequence

The motion sequence is structured around a coordinated set of approach, pick, assembly, and placement positions:

- **Cylinder Points:** (`red1`, `appred1`) define the approach and grasp locations for the red cylinder at the dispenser.
- **Piston Points:** (`upblack1-4`, `black1-4`) define safe approach and pick locations for each piston in the source tray.
- **Assembly Points:** (`apptog1`, `together1`) are used to place the cylinder on the assembly station, while (`apptog2`, `together2`) define the insertion path for the piston.
- **Matrix Points:** (`upmatrix1-4`, `matrix1-4`) define the four storage positions where the final assemblies are deposited.

Each motion command is executed using `movej`, and every movement is followed by `waitEndMove()` to ensure that the gripper operation or subsequent motion is initiated only after the robot has stopped.

I/O Operations

- **Outputs:**
  - `gripper` – Digital output controlling the pneumatic gripper (open/close).

- `dispenserAct` – Controls the pneumatic actuator responsible for feeding red cylinders into the dispenser.

- **Inputs:**
  - `cylPresence` – Indicates whether a red cylinder is available at the picking location.
  - `toolPresence` – Mounted on the tool to detect pistons (used in Level 3 to skip empty tray positions).
  - `actExtended`, `actContracted` – Feedback sensors that confirm extension/retreat motion of the cylinder dispenser actuator.

Waiting and Synchronization

Synchronization between robot motion and hardware actuation is critical in this laboratory setting. The program ensures safe execution using:

- `waitEndMove()` to guarantee completion of each robot movement.
- `wait(condition)` to ensure the robot does not proceed until a sensor confirms the correct pneumatic position.
- Small timing offsets such as `delay(0.3)` following gripper operations to ensure stable gripping or releasing.

These synchronization steps ensure that cylinders are properly dispensed, pistons are accurately detected, and assemblies are placed reliably in the final matrix without collision or slippage.

Program Flow (Pseudocode)

Task Parameters

The parameters used in this assembly task include the robot speed configuration, Cartesian points for picking and placing components, tool frames for the two gripping orientations, and the digital I/O mapped to the cylinder dispenser and tool sensors.

Joint and Cartesian Positions

The robot uses a set of predefined approach and operation Cartesian points for:

- **Red Cylinder Handling:**
  - `red1`, `appred1` – approach and pick points at the cylinder dispenser.
  - `apptog1`, `together1` – placement into the assembly station.
- **Black Piston Handling:**
  - `upblack1-4`, `black1-4` – four piston locations in the piston tray.
  - `apptog2`, `together2` – approach and insertion positions above the assembled cylinder.

Algorithm 1 Assembly Procedure for Red Cylinder + Black Piston using VAL3

Initialize robot, enable power, load tool frames
Set nominal robot speed: `mNomSpeed.vel = 30`
Link all digital inputs/outputs using `dioLink()`
Move robot to safe home position and open both grippers
Display "System Ready – Starting Level 2 Assembly"
for $i = 1$ to 4 **do**

// — **FEED RED CYLINDER** —
Activate dispenser actuator: `dispenserAct = true`
Wait until actuator fully extended: `wait(actExtended == true)`
Retract actuator: `dispenserAct = false`
Wait until actuator is contracted: `wait(actContracted == true)`
Wait for cylinder ready sensor: `wait(cylPresence == true)`

// — **PICK RED CYLINDER** —
Move above cylinder: `movej(red1)`
Move to pick position: `movej(appred1)`
Close lateral gripper: `close(Tool0[0])`
Lift cylinder from dispenser: `movej(red1)`

// — **PLACE RED CYLINDER IN ASSEMBLY STATION** —
Move above assembly station: `movej(apptog1)`
Place cylinder: `movej(together1)`
Open lateral gripper: `open(Tool0[0])`
Lift away from assembly: `movej(apptog1)`

// — **PICK PISTON DEPENDING ON INDEX** i — Switchi
Case1

Move to piston position 1 (`upblack1` → `black1`) EndCase
Case2

Move to piston position 2 (`upblack2` → `black2`) EndCase
Case3

Move to piston position 3 (`upblack3` → `black3`) EndCase
Case4

Move to piston position 4 (`upblack4` → `black4`) EndCase
EndSwitch

Close tip gripper: `close(Tool0[1])`
Lift piston back to safe height

// — **INSERT PISTON INTO RED CYLINDER** —
Move above insertion point: `movej(apptog2)`

- **Matrix Storage:**
  - `upmatrix1-4`, `matrix1-4` – four placement points for final assemblies.

All coordinates were recorded using the teach pendant and refined incrementally during real-robot testing.

Tool Frames

Two tool frames were used:

- **Tool0[0]: Lateral Tool** – used for grasping the red cylinders.
- **Tool0[1]: Tip Tool** – used for picking the black pistons.

These tool frames define the orientation offset required for each gripping operation and ensure accurate approach paths during manipulation.

Robot Speed

All motions use the nominal velocity:

$$mNomSpeed.vel = 30$$

As required by the laboratory safety rules, initial testing was conducted at 10% speed from the MCP, then gradually increased to full speed once trajectories were verified to be safe.

Digital I/O (DIO)

The digital inputs and outputs used are:

- **Outputs:**
  - `bOut0`: gripper (open/close)
  - `bOut10`: cylinder dispenser pneumatic actuator
- **Inputs:**
  - `bIn0`: tool-mounted presence sensor (used in Level 3)
  - `bIn2`: cylinder presence sensor at dispenser
  - `bIn14`, `bIn15`: actuator extended/contracted feedback

All I/O were linked in the program using `dioLink()`, enabling real-time coordination between robot movements and external pneumatic devices.

Safety Considerations

The Stäubli TS60 SCARA robot is a high-speed industrial manipulator, and strict safety measures were implemented throughout the experiment. All robot trajectories were first validated in the SRS simulation environment to ensure that no collisions occurred with the cylinder dispenser, piston tray, assembly station, or matrix. The "stop on first collision" option was enabled during simulation to immediately detect unsafe paths before deploying them to the real robot.

On the real TX60 controller, the maximum speed was limited to `mNomSpeed.vel = 30`, and the first execution was performed at 10% speed using the MCP to ensure safe testing. Emergency stop buttons, the Move/Hold button, and the external STOP switch were identified and tested prior to execution.

Safe approach points (`upblack1-4`, `upmatrix1-4`, `appred1`, `apptog1`, `apptog2`) were defined to guarantee that the robot always moved vertically before entering or exiting any constrained area. All gripper actions, pneumatic actuations, and sensor feedback signals were verified through `dioLink()` to avoid unintended motion. This ensured safe and reliable operation across all stages of the pick-and-place cycle.

Execution Results

The TS60 robot successfully executed the complete Level 2 routine: feeding red cylinders using the dispenser actuator, picking them with the lateral tool, placing them in the assembly station, picking pistons from the fixed piston tray order (positions 1 to 4), inserting them into the cylinders, and finally placing the assembled components into the correct matrix positions.

All motions were executed smoothly using joint movements (`movej`) with synchronized gripper actions (`open`/`close`). The program handled all four assemblies without collisions or unexpected behavior. The tool presence sensor was intentionally not used in Level 2, and the system relied solely on a fixed piston order, producing consistent and repeatable results.

The overall cycle time varied depending on the travel distance to each piston and matrix position, but all assemblies were completed correctly, demonstrating stable performance of the robot and proper coordination between I/O actions and robot movement.

Improvements

Several enhancements could further optimize the system:

Reducing cycle time:
The robot speed could be increased beyond the nominal 30 (within safety limits), and the delay values between gripper actions could be optimized. Using blended motions instead of strictly sequential `waitEndMove()` calls may also shorten execution time.

Minimizing robot travel distance:
Reordering the piston picking or matrix placement sequence (e.g., picking the closest piston first) could reduce unnecessary movement and shorten cycle time.

Tool and gripping optimization:
Fine-tuning the gripping force, tool offsets, or approach heights may reduce time spent aligning with the cylinders and pistons.

Collision and safety optimization:
Refining the approach and retreat points could further increase clearance from the assembly station and tray edges while maintaining efficient paths.

Automation and sensing improvements:
Adding adaptive logic—such as using the tool presence sensor (as in Level 3)—would allow the robot to skip empty piston positions instead of following a fixed sequence, improving flexibility and real-world robustness.